DSA LAB PROGRAMS

C Programming – Data Structures and Algorithms

Programs arranged according to the priority/order in the provided lab list.

1. STACK OPERATIONS

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push()
{
    int value;

    if (top == MAX - 1)
    {
        printf("Stack Overflow!\n");
    }
    else
    {
        printf("Enter value: ");
        scanf("%d", &value);

        top++;
        stack[top] = value;

        printf("%d pushed into stack.\n", value);
    }
}

void pop()
{
    if (top == -1)
    {
        printf("Stack Underflow!\n");
    }
    else
    {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}

void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is empty.\n");
    }
    else
    {
        printf("Stack elements are:\n");

        for (i = top; i >= 0; i--)
            printf("%d\n", stack[i]);
    }
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n--- STACK OPERATIONS ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

2. INFIX TO POSTFIX CONVERSION

```
#include <stdio.h>
#include <ctype.h>

char stack[100];
int top = -1;

void push(char x)
{
    stack[++top] = x;
}

char pop()
{
    if (top == -1)
        return '\0';

    return stack[top--];
}

int priority(char x)
{
    if (x == '(')
        return 0;

    if (x == '+' || x == '-')
        return 1;

    if (x == '*' || x == '/' || x == '%')
        return 2;

    if (x == '^')
        return 3;

    return -1;
}

int main()
{
    char exp[100];
    char *e;

    printf("Enter infix expression: ");
    scanf("%s", exp);

    e = exp;

    printf("Postfix expression: ");

    while (*e != '\0')
    {
        if (isalnum((unsigned char)*e))
        {
            printf("%c", *e);
        }
        else if (*e == '(')
        {
            push(*e);
        }
        else if (*e == ')')
        {
            while (top != -1 && stack[top] != '(')
                printf("%c", pop());

            if (top != -1 && stack[top] == '(')
                pop();
        }
        else
        {
            while (top != -1 &&
                   stack[top] != '(' &&
```

```
                priority(stack[top]) >= priority(*e))
            {
                printf("%c", pop());
            }

            push(*e);
        }

        e++;
    }

    while (top != -1)
        printf("%c", pop());

    printf("\n");

    return 0;
}
```

3. BINARY SEARCH – RECURSIVE & NON-RECURSIVE

```
#include <stdio.h>

/* Non-Recursive Binary Search */
int binarySearchNonRecursive(int a[], int n, int key)
{
    int low = 0;
    int high = n - 1;
    int mid;

    while (low <= high)
    {
        mid = low + (high - low) / 2;

        if (a[mid] == key)
            return mid;
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    return -1;
}

/* Recursive Binary Search */
int binarySearchRecursive(int a[], int low, int high, int key)
{
    int mid;

    if (low > high)
        return -1;

    mid = low + (high - low) / 2;

    if (a[mid] == key)
        return mid;
    else if (key < a[mid])
        return binarySearchRecursive(a, low, mid - 1, key);
    else
        return binarySearchRecursive(a, mid + 1, high, key);
}

int main()
{
    int a[] = {10, 20, 30, 40, 50, 60, 70};
    int n = 7;
    int key = 40;
    int result;

    result = binarySearchNonRecursive(a, n, key);

    if (result != -1)
        printf("Non-Recursive: Element found at index %d\n", result);
    else
        printf("Non-Recursive: Element not found\n");

    result = binarySearchRecursive(a, 0, n - 1, key);

    if (result != -1)
        printf("Recursive: Element found at index %d\n", result);
    else
        printf("Recursive: Element not found\n");

    return 0;
}
```

4. TOWER OF HANOI

```
#include <stdio.h>

void tower(int n, char source, char destination, char auxiliary)
{
    if (n == 1)
    {
        printf("Move disk 1 from %c to %c\n",
               source, destination);
        return;
    }

    tower(n - 1, source, auxiliary, destination);

    printf("Move disk %d from %c to %c\n",
           n, source, destination);

    tower(n - 1, auxiliary, destination, source);
}

int main()
{
    int n = 3;

    tower(n, 'A', 'C', 'B');

    return 0;
}
```

5. QUEUE OPERATIONS

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue()
{
    int value;

    if (rear == MAX - 1)
    {
        printf("Queue Overflow!\n");
    }
    else
    {
        printf("Enter value: ");
        scanf("%d", &value);

        if (front == -1)
            front = 0;

        rear++;
        queue[rear] = value;

        printf("%d inserted into queue.\n", value);
    }
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow!\n");
    }
    else
    {
        printf("%d deleted from queue.\n", queue[front]);
        front++;

        if (front > rear)
        {
            front = -1;
            rear = -1;
        }
    }
}

void display()
{
    int i;

    if (front == -1)
    {
        printf("Queue is empty.\n");
    }
    else
    {
        printf("Queue elements are:\n");

        for (i = front; i <= rear; i++)
            printf("%d ", queue[i]);

        printf("\n");
    }
}

int main()
```

```
{
    int choice;

    while (1)
    {
        printf("\n--- QUEUE OPERATIONS ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```